

Puerta Inteligente para Mascotas con Control Adaptativo de Iluminación

Sistema Embebido basado en ESP32 con FreeRTOS

Bianchi, Juan Alberto; Brocani, Agustín Nicolás; Ghiano, Gonzalo Agustín; Martucci, Federico Ariel; Minaudo, Lucas

DNI 30.475.902 - DNI 40.931.870 - DNI 43.448.036 - DNI 44.690.247 - DNI 43.316.993

Martes - Grupo M1

Universidad Nacional de La Matanza,
Departamento de Ingeniería e Investigaciones Tecnológicas,
Florencio Varela 1903 - San Justo, Argentina

Resumen. El presente trabajo describe el diseño y la implementación de un sistema embebido orientado a Internet de las Cosas (IoT), construido sobre una placa ESP32 simulada en Wokwi y programada con el sistema operativo FreeRTOS. El sistema implementa una puerta inteligente para mascotas que permite el ingreso desde el exterior mediante identificación por tarjeta RFID y la salida desde el interior mediante un sensor de proximidad ultrasónico, accionando un servomotor en distintas direcciones según el sentido de paso. De forma paralela, un fototransistor monitorea la luz ambiente para encender automáticamente una luminaria cuando se detecta oscuridad. El bloqueo y desbloqueo manual de la puerta se realiza mediante un pulsador que simula un comando externo enviado desde una aplicación, generando además una señalización sonora a través de un buzzer. La arquitectura aplica el patrón de máquina de estados con tareas concurrentes y colas para los eventos y las acciones.

Palabras claves: sistema embebido, ESP32, FreeRTOS, máquina de estados, IoT.

1 Introducción

Los sistemas embebidos aplicados a Internet de las Cosas (IoT) permiten automatizar tareas domésticas mediante la coordinación de sensores y actuadores que comparten información en tiempo real. En este trabajo se desarrolla un prototipo de puerta inteligente para mascotas, capaz de identificar al animal cuando éste se aproxima desde el exterior, permitir su salida cuando se desplaza desde el interior y, simultáneamente, regular el encendido de una luminaria en función de la iluminación ambiente.

El sistema fue construido sobre una placa ESP32 simulada en el entorno Wokwi y programado en C++ con el sistema operativo FreeRTOS, lo que permite organizar el comportamiento en tareas concurrentes que se comunican entre sí mediante colas. Se aplica el patrón de diseño máquina de estados, una tabla de transiciones y temporizadores no bloqueantes, evitando el uso de funciones como delay().

URL del proyecto en Wokwi: <https://wokwi.com/projects/463205516744945665>

2 Desarrollo

2.1 Descripción funcional del sistema

El sistema embebido se modela como una única máquina de estados finita que integra el control de la puerta y el control de iluminación, con su tabla de transiciones, sus colas de eventos y de acciones, y sus tareas FreeRTOS asociadas.

El control de iluminación lee periódicamente el valor analógico del fotorresistor y, en función de un umbral predefinido, enciende o apaga una luminaria representada por un LED. El control de la puerta combina dos sensores —un módulo RFID MFRC522 para identificar a la mascota desde el exterior y un sensor ultrasónico HC-SR04 para detectar su presencia desde el interior— con un servomotor que abre la puerta en una u otra dirección según corresponda. Adicionalmente, un pulsador físico emula el comando que enviaría una aplicación móvil para bloquear o desbloquear la puerta, y un buzzer genera una señal sonora distintiva en cada operación, dependiendo de si el sistema bloquea la puerta o la desbloquea.

2.2 Diagrama de estados

El sistema implementa una única máquina de estados que integra el comportamiento de la puerta y el control de iluminación. La máquina contempla cinco estados (arranque, cerrada no bloqueada, cerrada bloqueada, abierta desde afuera y abierta desde adentro) gobernados por nueve eventos: `init_no_bloqueado` e `init_bloqueado` para la transición inicial desde arranque, `animal_detectado_afuera` y `animal_detectado_adentro` para las aperturas, `timeout` para el cierre automático, `bloqueo_por_app` y `desbloqueo_por_app` para la alternancia manual, y `noche_detectada` y `dia_detectado` que comandan el encendido y apagado de la luminaria sin alterar el estado de la puerta. Cada cambio de estado se registra por consola serial, lo que permite trazar la ejecución durante la simulación.

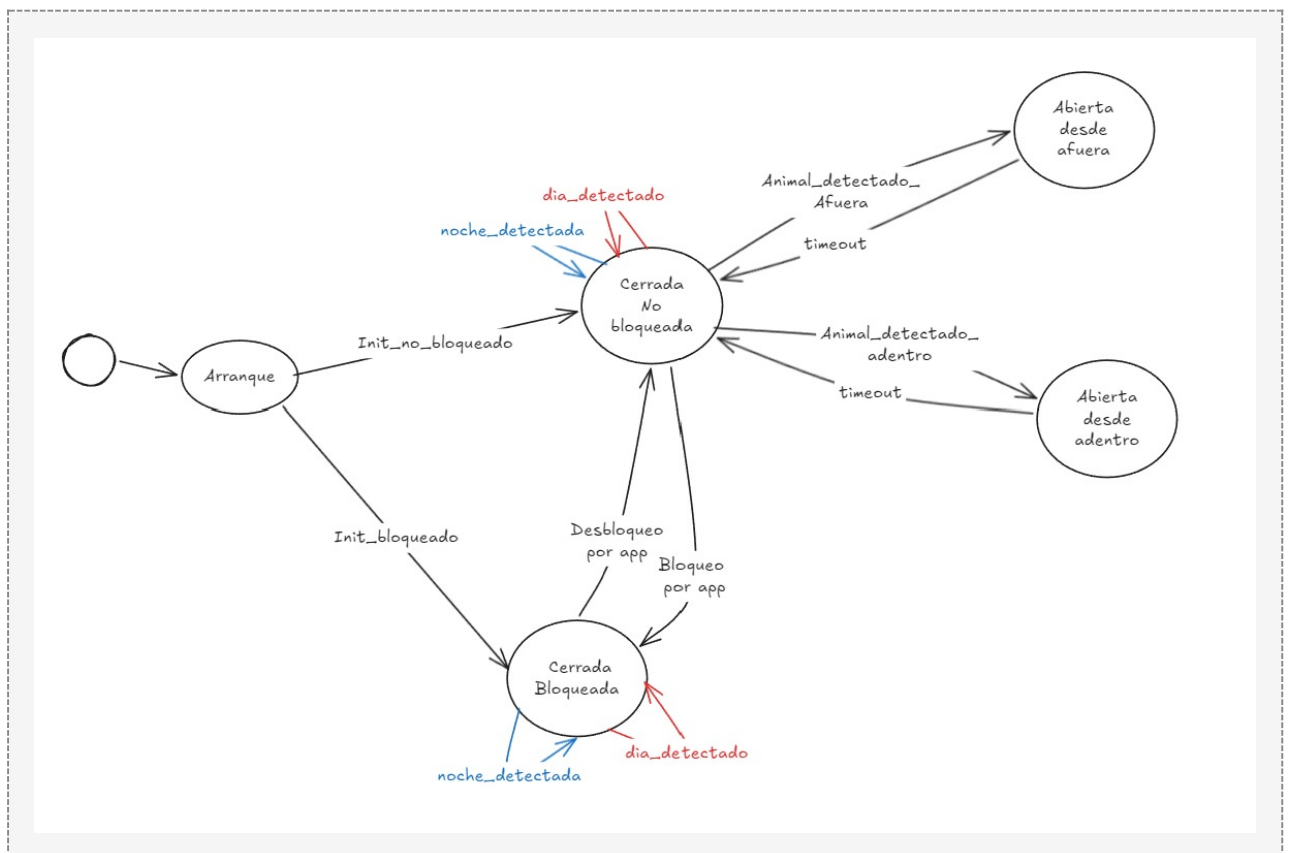


Figura 1. Diagrama de estados del sistema embebido.

2.3 Diagrama de conexiones del circuito

El circuito se construyó respetando el estándar de colores (rojo para alimentación positiva, negro para masa) y empleando verde y violeta para las conexiones entre sensores y actuadores con la placa. Las conexiones se trazaron en forma horizontal o vertical sobre la protoboard, y el conjunto se alimenta desde una fuente regulable externa de 5 V para los componentes del sistema que necesitan este tipo de voltaje.

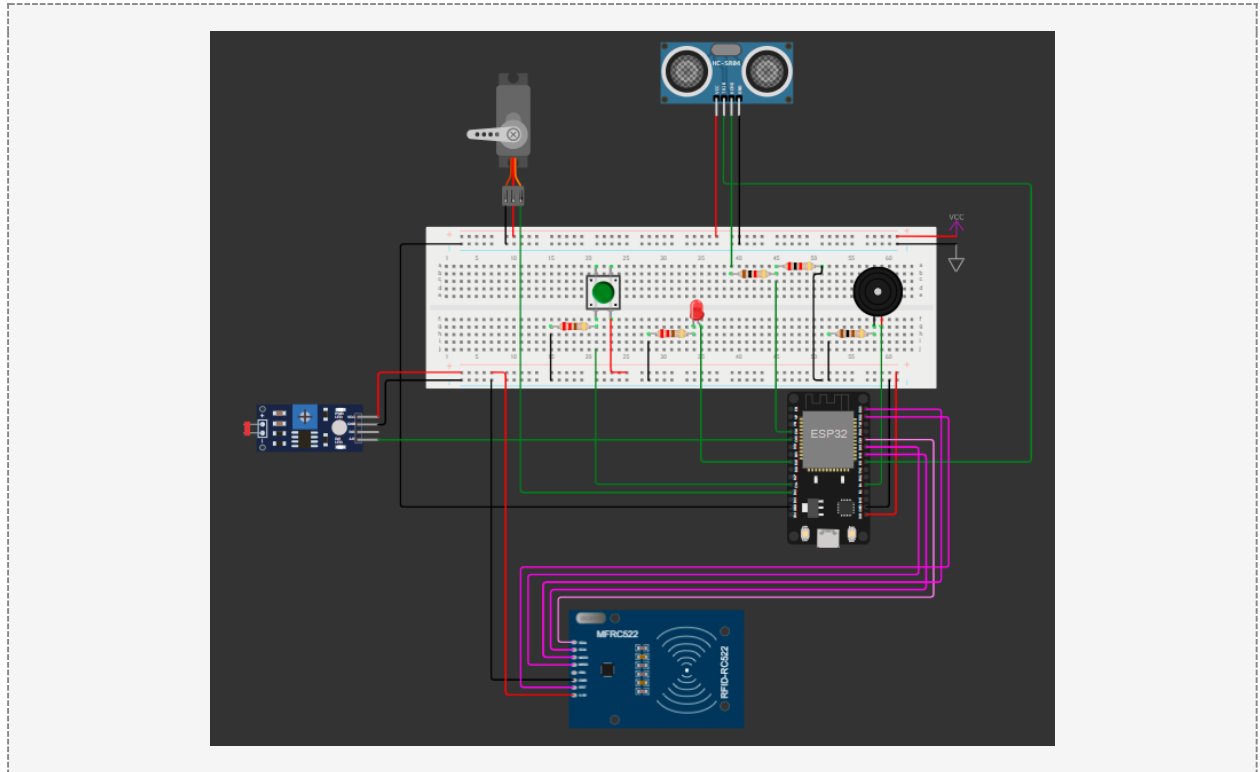


Figura 2. Diagrama de conexiones del circuito en Wokwi.

2.4 Descripción físico-electrónica de sensores y actuadores

A continuación, se detalla el principio de funcionamiento de cada componente utilizado, así como su rol dentro del sistema y los pines de la ESP32 a los que se encuentra conectado.

a) Fotorresistor (LDR) — sensor analógico de luz.

El fotorresistor es una resistencia variable cuyo valor disminuye al aumentar la intensidad de luz incidente sobre su superficie semiconductor de sulfuro de cadmio. Conectado en un divisor de tensión, entrega una tensión analógica que se digitaliza mediante el ADC de la ESP32 a través del pin GPIO 35. Cuando el valor leído supera el umbral definido (UMBRAL_LUZ) se considera que es de noche y se emite el evento correspondiente a la cola de eventos de la máquina de estados.

b) Sensor ultrasónico HC-SR04 — sensor digital de proximidad.

Este sensor mide la distancia a un objeto mediante la emisión de un pulso ultrasónico de 40 kHz por su transductor TRIG y la recepción del eco reflejado por el transductor ECHO. El tiempo transcurrido entre la emisión y la recepción se traduce a distancia utilizando la velocidad del sonido ($0,0343 \text{ cm}/\mu\text{s}$). El sensor se conecta a la ESP32 con TRIG en GPIO 5 y ECHO en GPIO 34. Si la distancia medida resulta inferior a un mínimo configurable (30 cm) y la puerta se encuentra en estado cerrada-no-bloqueada, se interpreta como la presencia de la mascota desde el interior y se dispara el evento de apertura desde adentro.

c) Módulo RFID MFRC522 — sensor digital de identificación.

El módulo MFRC522 utiliza un campo electromagnético de 13,56 MHz para alimentar y comunicarse con tarjetas RFID pasivas mediante el protocolo ISO/IEC 14443A. La comunicación con la ESP32 se realiza por bus SPI

(SCK en GPIO 18, MOSI en GPIO 23, MISO en GPIO 19, SS en GPIO 21 y RST en GPIO 22). Cuando la mascota se aproxima desde el exterior con su collar identificador, el lector detecta la tarjeta, valida su presencia y dispara el evento de apertura desde afuera, siempre que la puerta esté cerrada y no bloqueada.

d) Pulsador — entrada digital de control.

El pulsador conectado al GPIO 14 con resistencia de pull-down externa simula el comando que enviaría una aplicación móvil para bloquear o desbloquear la puerta. Cada pulsación alterna el estado lógico (toggle) y emite a la cola el evento EV_BLOQUEO_POR_APP o EV_DESBLOQUEO_POR_APP según corresponda.

e) LED — actuador digital.

El LED conectado al GPIO 25 funciona como luminaria y se controla por escritura digital alta o baja. Su encendido y apagado depende exclusivamente de las acciones emitidas por la máquina de estados, lo que asegura que la lectura del sensor no actúe directamente sobre el actuador, respetando el patrón productor-consumidor.

f) Servomotor SG90 — actuador con PWM.

El servomotor se controla mediante una señal PWM de 50 Hz cuyo ciclo de trabajo determina la posición angular del eje. La biblioteca ESP32Servo abstrae el cálculo del pulso y permite escribir la posición deseada en grados sobre el GPIO 12. La posición de reposo (puerta cerrada) corresponde a 90°, la apertura desde adentro corresponde a 180° y la apertura desde afuera a 0°. Tras un timeout de 4,5 segundos gestionado por un xTimerCreate de FreeRTOS, la puerta retorna automáticamente a la posición cerrada.

g) Buzzer piezoeléctrico — actuador sonoro.

El buzzer se conecta al GPIO 4 y suena mediante una onda cuadrada generada por software (alternando digitalWrite con delayMicroseconds) para evitar el conflicto entre el periférico LEDC y la biblioteca ESP32Servo, que comparten el mismo timer interno. Esta solución permite emitir tonos de frecuencia y duración variables: un barrido descendente de 600 Hz a 300 Hz para el bloqueo y uno ascendente de 600 Hz a 1200 Hz para el desbloqueo, otorgando una retroalimentación auditiva clara al usuario.

2.5 Arquitectura de software

2.5.1 Embebido

El firmware se organiza siguiendo el patrón productor-consumidor mediante colas FreeRTOS. La máquina de estados se sostiene sobre tres tareas con la misma prioridad: una tarea de detección que muestrea los sensores y emite eventos, una tarea controladora que recibe los eventos, ejecuta la transición correspondiente de la tabla de estados y emite acciones, y una tarea de acción que recibe las acciones y manipula los actuadores. Esta separación garantiza que las funciones de lectura y escritura nunca se ejecuten en el mismo hilo, evitando bloqueos.

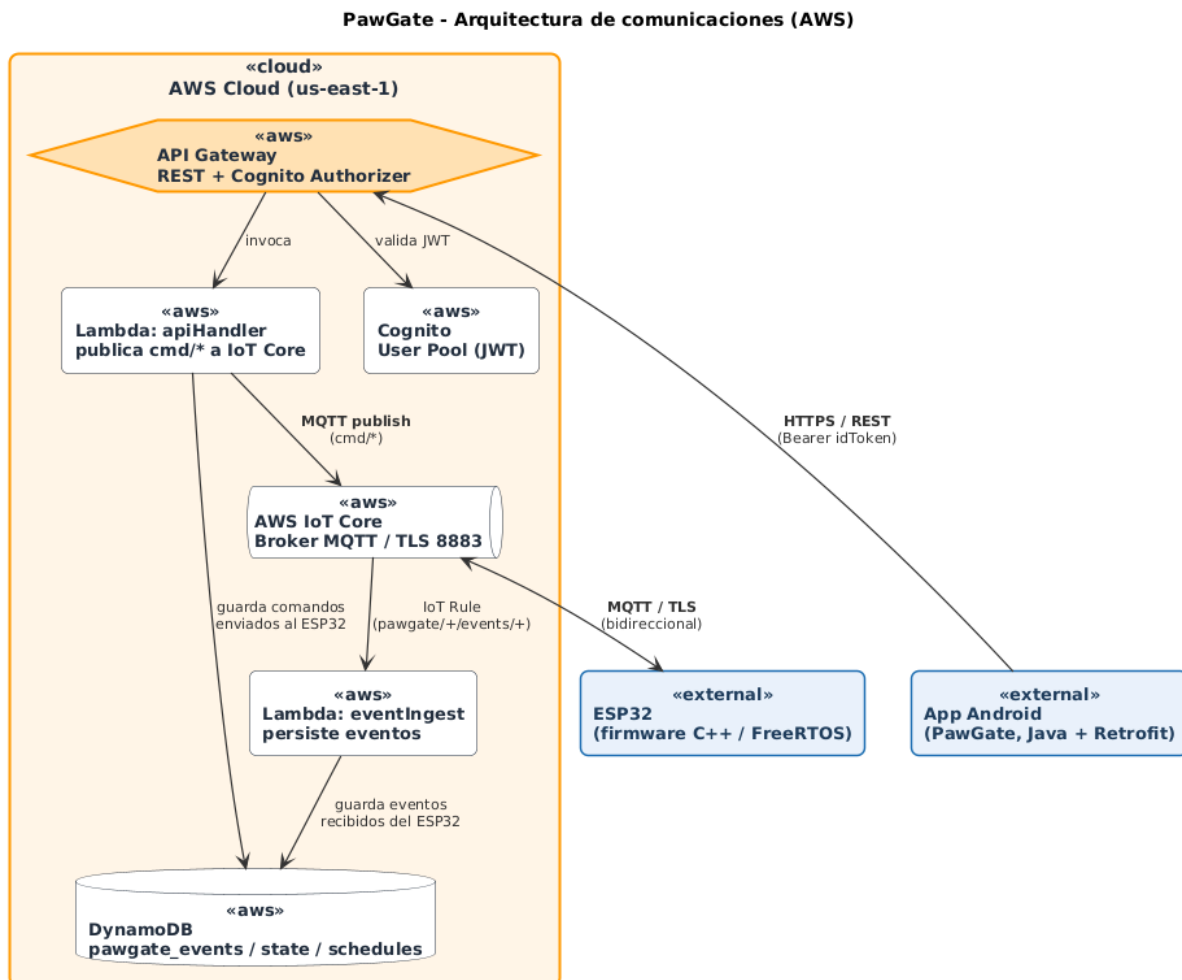
La tabla de estados se implementa como una matriz bidimensional de punteros a función (transición `tabla_estados[CANT_MAX_ESTADOS][CANT_MAX_EVENTOS]`), donde cada celda contiene el manejador correspondiente a un par estado-evento. Esto elimina el uso de extensos bloques switch-case anidados y vuelve declarativo el comportamiento del autómata. Todas las constantes están parametrizadas mediante macros (`#define`) ubicadas al inicio del archivo para evitar cualquier tipo de uso de magic numbers.

Los retardos se manejan exclusivamente con `vTaskDelay(pdMS_TO_TICKS(...))` para no bloquear el planificador. El timeout de cierre automático de la puerta se gestiona con un timer por software (`xTimerCreate`) que dispara un callback al expirar, emitiendo el evento EV_TIMEOUT a la cola de eventos.

2.5.2 AWS como proveedor de infraestructura en la Nube

Dentro de la infraestructura de AWS se debe destacar la existencia del Broker de MQTT con el servicio “AWS IOT CORE” el cual nos permite la comunicación vía tópicos desde el embebido y con la aplicación móvil. El embebido se comunica bidireccionalmente hacia los tópicos directamente con MQTT, en cambio la APP se comunica vía API REST hacia unas funciones Lambdas cargadas en la nube, las cuales hacen la traducción del MQTT del broker y la response necesaria por el llamado de la API.

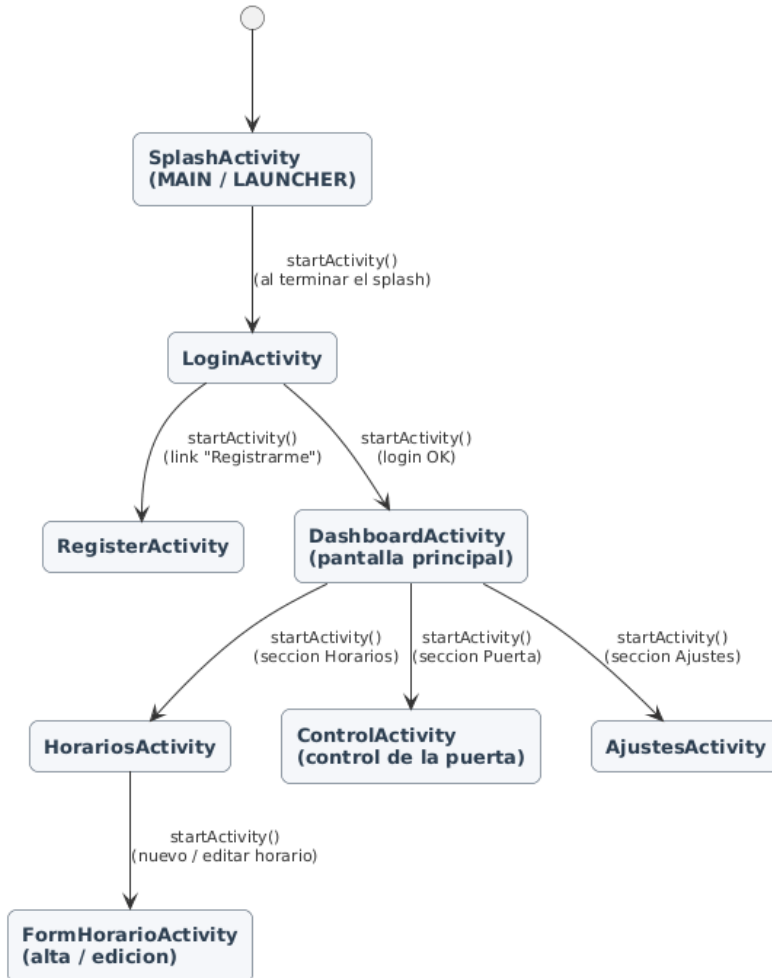
En el siguiente diagrama se muestra de manera básica la infraestructura utilizada en la nube, junto a los protocolos de comunicación utilizados:



2.6 Diagrama de navegación

Se adjunta el diagrama de navegación de las activities de la aplicación de Android “Pawgate” que muestra que pantallas redirigen a otras pantallas a través de las acciones que realizan los usuarios.

PawGate - Navegacion entre Activities (flujo principal)



A modo de resumen, el flujo principal para tener presente en el día a día sería:

1. El usuario inicia la aplicación entrando a la *SplashActivity*.
2. Pasa al *LoginActivity* e inicia sesión a la aplicación, (en caso de ya estar logueado esta vista se saltea).
3. Entra al dashboard donde tiene toda la información relevante acerca del dispositivo y presiona bloquear.
4. Al bloquear lo envía a la *ControlActivity* (visualizada en la APP como “Puerta”) donde le muestra el bloqueo de la puerta.

Otro flujo de interés para el usuario sería la configuración de horarios (dando 1. y 2. por hechos):

3. Entra al dashboard donde tiene toda la información relevante acerca del dispositivo y presiona horarios.
4. El sistema abre *HorariosActivity* donde se visualizan todos los horarios configurados y su información.
5. Presiona Agregar horario y el sistema abre *FormHorarioActivity*.
6. El usuario ingresa toda la información relevante del nuevo horario a configurar y guarda una vez completada la información.

Estos flujos se analizan en profundidad en el Manual de Usuario, sección 2.7 de esto informe.

2.7 Manual de usuario

2.7.1 Embebido

El presente manual describe paso a paso cómo poner en funcionamiento el prototipo y verificar la totalidad de sus funcionalidades, tanto desde la simulación en Wokwi como desde el monitor serial integrado.

Paso 1. Encendido del sistema.

Al iniciar la simulación, la ESP32 ejecuta la rutina `setup()`, inicializa los pines, las colas, las tareas y los temporizadores. La máquina parte en estado cerrada-no-bloqueada (servo en 90°) con la luminaria apagada. El monitor serial debe configurarse a 115200 baudios para visualizar la traza de eventos y acciones.

Paso 2. Funcionamiento del control de iluminación.

El usuario puede modificar el valor del fotorresistor desde la interfaz de Wokwi. Si la lectura supera el umbral configurado, el sistema imprime '>> Evento emitido: EV_NOCHE_DETECTADA' y enciende el LED. Si la lectura cae por debajo del umbral, el sistema imprime '>> Evento emitido: EV_DIA_DETECTADO' y apaga el LED. La transición es automática y no requiere intervención del usuario.

Paso 3. Apertura desde el interior (sensor ultrasónico).

Con la puerta cerrada y no bloqueada, al disminuir la distancia simulada del HC-SR04 por debajo de 30 cm el sistema emite el evento `EV_ANIMAL_DETECTADO_ADENTRO` y el servomotor gira a 180°. Tras 4,5 segundos, el timer dispara el evento `EV_TIMEOUT` y la puerta retorna a 90°. Durante el tiempo en que la puerta está abierta, ambos sensores quedan deshabilitados para evitar disparos en momentos no deseados.

Paso 4. Apertura desde el exterior (RFID).

Con la puerta cerrada y no bloqueada, al acercar una tarjeta RFID válida al lector MFRC522 el sistema emite el evento `EV_ANIMAL_DETECTADO_AFUERA` y el servomotor gira a 0°. El comportamiento de cierre automático tras 4,5 segundos es idéntico al del caso anterior.

Paso 5. Bloqueo y desbloqueo desde la aplicación.

Para bloquear las aperturas de la puerta se debe presionar el pulsador conectado al GPIO 14, lo que alterna entre los estados bloqueada y no bloqueada. Cada operación produce una señal acústica distintiva: tono descendente para el bloqueo y tono ascendente para el desbloqueo. Mientras la puerta esté bloqueada, ningún sensor podrá abrirla.

Paso 6. Verificación por monitor serial.

Todas las transiciones de estado se imprimen en la consola serial con el formato '[funcion] Mensaje', lo que permite verificar la coherencia entre los eventos de entrada, las transiciones ejecutadas y las acciones aplicadas sobre los actuadores.

2.7.2 Aplicación de Android “PawGate”

La aplicación Android "PawGate" es el punto de acceso remoto al dispositivo. El presente apartado describe, paso a paso, los dos flujos principales de uso: el bloqueo y desbloqueo remoto de la puerta desde la vista Control, y la creación de un horario de bloqueo automático. Todos los flujos parten de la apertura de la aplicación desde el ícono del launcher, momento en el cual el sistema muestra la pantalla *SplashActivity* durante un breve intervalo antes de derivar al usuario a la pantalla de autenticación.

Flujo 1. Bloqueo y desbloqueo remoto de la puerta desde la aplicación

Paso 1. Ingreso a la aplicación.

Al abrir "PawGate", el usuario visualiza la pantalla *SplashActivity*, que actúa como pantalla de bienvenida. Transcurrido el retardo de arranque, la aplicación evalúa si existe una sesión vigente en el almacenamiento local (JWT válido) y, en caso negativo, transiciona automáticamente a *LoginActivity*.

Paso 2. Autenticación del usuario.

En *LoginActivity* el usuario ingresa su correo electrónico y contraseña, y presiona el botón "Ingresar". La aplicación valida las credenciales contra el servicio de autenticación. Si la autenticación es exitosa, el sistema persiste el token y navega a *DashboardActivity*; si falla, se muestra un mensaje de error debajo de los campos. Desde esta misma pantalla, si el usuario aún no tiene cuenta, puede tocar el enlace "Registrarme" para abrir *RegisterActivity*.

Paso 3. Pantalla principal (Dashboard).

Una vez autenticado, la aplicación despliega *DashboardActivity*, la vista principal, que resume el estado actual de la puerta ("Cerrada", "Abierta", "Bloqueada"), las aperturas del día, la última actividad y el estado de la luminaria. En la parte inferior de la pantalla se presenta la barra de navegación con cuatro pestañas: Inicio, Puerta, Historial y Ajustes.

Paso 4. Acceso a la vista Control.

Para operar el bloqueo remoto, el usuario debe tocar la pestaña "Puerta" de la barra de navegación inferior. La aplicación abre *ControlActivity*, la cual muestra el estado actual del dispositivo, un botón principal "ABRIR" y dos tarjetas de acción rápida: "Bloquear" y "Llamar".

Paso 5. Bloqueo de la puerta.

El usuario presiona la tarjeta "Bloquear". La aplicación consulta a los servidores si en ese momento existe un horario activo y, en base a esa información, muestra un cuadro de diálogo de confirmación. Al confirmar, la interfaz aplica el cambio, envía el comando `cmd/block` a los servidores y este lo publica al tópico MQTT correspondiente que lo comunica con el embebido. El firmware del ESP32 recibe el comando, transiciona al estado bloqueada y emite el tono descendente de bloqueo. En la aplicación, el estado del encabezado pasa a "BLOQUEADA", el botón principal se deshabilita, la tarjeta "Bloquear" queda destacada como activa y aparece la leyenda "Modo seguridad activado".

Paso 6. Desbloqueo desde la misma vista.

Con la puerta ya bloqueada, *ControlActivity* habilita en la zona inferior un botón secundario rotulado "Desbloquear". Al presionarlo, la aplicación evalúa nuevamente el contexto de horario y muestra el diálogo de confirmación apropiado. Confirmada la acción, envía el comando `cmd/unblock`; el firmware transiciona al estado no bloqueada, emite el tono ascendente de desbloqueo y la aplicación regresa al estado "Cerrada" en la vista Control, quedando disponible nuevamente el botón "ABRIR". El usuario puede volver a *DashboardActivity* presionando la pestaña "Inicio" de la barra de navegación.

Flujo 2. Creación de un horario de bloqueo automático

Paso 1. Ingreso a la aplicación y autenticación.

El usuario abre "PawGate", atraviesa la pantalla *SplashActivity* e inicia sesión desde *LoginActivity* según lo descrito en el Paso 2 del Flujo 1. La aplicación deriva a *DashboardActivity*.

Paso 2. Acceso a la sección Horarios.

En *DashboardActivity*, el usuario presiona la tarjeta de acceso rápido "Horarios". La aplicación abre *HorariosActivity*, que muestra el listado de horarios existentes obtenido desde el backend. Si aún no existe ningún horario configurado, se presenta un estado vacío con la leyenda correspondiente y un botón "Agregar horario".

Paso 3. Alta de un nuevo horario.

El usuario presiona el botón "+" ubicado en el encabezado o el botón "Agregar horario" del estado vacío. La aplicación abre *FormHorarioActivity* en modo "Nuevo horario".

Paso 4. Ingreso del nombre.

El usuario completa el campo "Nombre" (por ejemplo, "Noche" o "Trabajo") mediante el teclado. El campo exige un mínimo de tres caracteres; si el requisito no se cumple, la aplicación muestra un error inline al intentar guardar.

Paso 5. Selección del rango horario.

El usuario toca sucesivamente los campos "Hora inicio" y "Hora fin". Cada acción abre un componente *MaterialTimePicker* en formato de reloj de 24 horas. Al confirmar la selección, la aplicación ajusta automáticamente los minutos al múltiplo de 30 más cercano (00 o 30) y, en caso de haber aplicado el ajuste, muestra un aviso al usuario. Se admite el cruce de medianoche (por ejemplo, 22:00 a 06:00), pero se rechaza que la hora de inicio y de fin sean idénticas.

Paso 6. Selección de días de la semana.

Debajo del rango horario, el formulario presenta siete chips con las iniciales "L M M J V S D". El usuario toca cada chip para incluirlo o excluirlo del horario. Se exige seleccionar al menos un día; si no lo hace, la aplicación muestra el error correspondiente al guardar.

Paso 7. Estado del horario.

El interruptor "Activo" permite indicar si el horario debe aplicarse desde su creación o quedar deshabilitado a la espera de una activación futura. Por defecto queda activo.

Paso 8. Persistencia del horario.

Finalizada la carga, el usuario presiona el botón "Guardar". La aplicación valida los tres criterios (nombre, días y horas), y de superarlos, se persiste en los servidores esa información. Ante una respuesta exitosa, muestra el mensaje "Horario creado" y regresa a *HorariosActivity*, donde el nuevo horario aparece en la lista. En caso de error, presenta el mensaje devuelto por el backend y permite reintentar. Si el usuario desea abandonar sin guardar, dispone del botón "Cancelar" o de la flecha de retorno en el encabezado.

Paso 9. Efecto sobre el dispositivo.

Una vez guardado el horario, el backend evalúa periódicamente los rangos configurados. Al ingresar en la ventana definida, publica un comando de bloqueo al tópico MQTT del dispositivo, y el ESP32 aplica el bloqueo del mismo modo que en el Flujo 1, sin requerir intervención adicional del usuario. Al finalizar la ventana, el desbloqueo se realiza automáticamente. Este comportamiento se refleja en tiempo real en *DashboardActivity* y *ControlActivity* a través del polling y las notificaciones push.

2.8 Implementación física

Se incluye a continuación una imagen del montaje físico realizado con todos los componentes adquiridos y las conexiones implementadas sobre la protoboard. El montaje respeta los mismos pines y el mismo cableado que el circuito simulado, lo que garantiza la equivalencia funcional entre ambas versiones del prototipo.

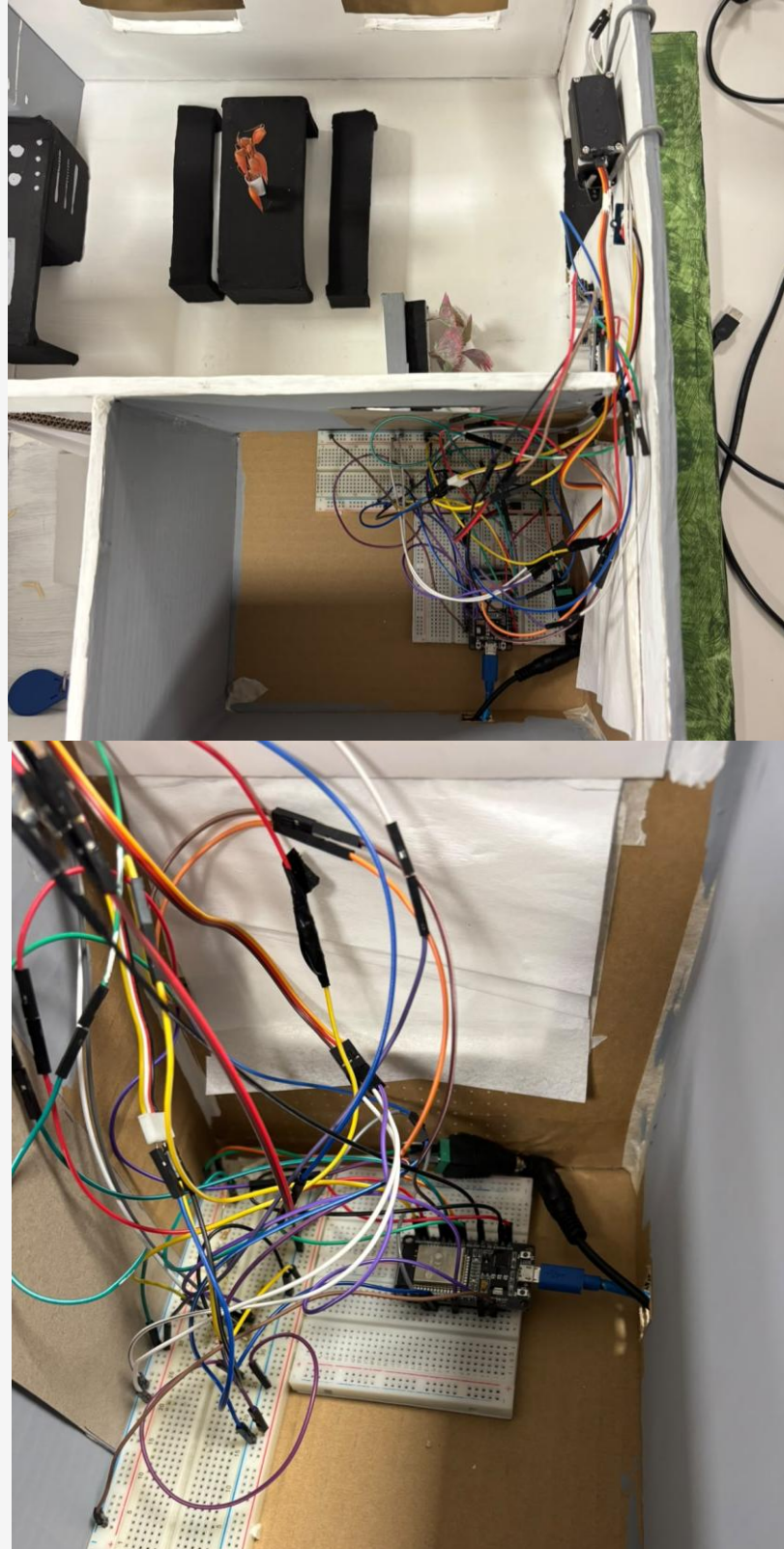


Figura 3. Implementación física del sistema embebido.

3 Referencias

- [1] Espressif Systems, “ESP32 Series Datasheet,” Rev. 4.6, Espressif Systems, Shanghai, China, 2024. [Online]. Disponible en: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
- [2] R. Barry, “Mastering the FreeRTOS Real Time Kernel: A Hands-On Tutorial Guide,” Real Time Engineers Ltd., 2016. [Online]. Disponible en: https://www.freertos.org/Documentation/RTOS_book.html
- [3] NXP Semiconductors, “MFRC522 — Standard 3V MIFARE reader solution, Product Data Sheet,” Rev. 3.9, NXP, Eindhoven, Países Bajos, 2016.
- [4] Cytron Technologies, “HC-SR04 Ultrasonic Sensor Datasheet,” Cytron Technologies, Pulau Pinang, Malasia, 2013.
- [5] Wokwi, “Wokwi — Online ESP32, STM32, Arduino Simulator,” 2024. [Online]. Disponible en: <https://wokwi.com/>
- [6] K. Yiu, “FreeRTOS Real-Time Kernel — A Hands-On Tutorial Guide for the ESP32,” Real Time Engineers Ltd., 2020. [Online]. Disponible en: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/system/freertos.html>
- [7] Cátedra de Sistemas Operativos Avanzados, “Apunte: Electrónica y Arduino en Tinkercad,” Universidad Nacional de La Matanza, San Justo, Argentina, 2024. [Online]. Disponible en: http://soa-unlam.com.ar/material-clase/Sistemas%20Embebidos/Apuntes%20Catedra/Apunte_Electronica_y_Arduino_en_Tinkercad.pdf